

Java Generics and Type Safety

1. What are Generics?

Generics allow you to define classes, interfaces, and methods with a type parameter.

Instead of:

```
List list = new ArrayList();

list.add("Payment");
list.add(100);
```

use:

```
List<String> list = new ArrayList<>();

list.add("Payment");
// list.add(100); // Compile-time error
```

The compiler knows that this list should contain only `String`.

2. Why do we need Generics?

Main benefits:

1. Type safety

Errors are detected during compilation.

```
List<String> names = new ArrayList<>();

names.add("Avnish");
names.add(100); // Compile-time error
```

2. No unnecessary casting

Without generics:

```
List list = new ArrayList();
list.add("Payment");

String value = (String) list.get(0);
```

With generics:

```
List<String> list = new ArrayList<>();
list.add("Payment");

String value = list.get(0);
```

3. Reusable code

The same generic class/method can work with different types.

3. Generic Class

Example:

```
public class Box<T> {  
    private T value;  
  
    public void set(T value) {  
        this.value = value;  
    }  
  
    public T get() {  
        return value;  
    }  
}
```

Now:

```
Box<String> box = new Box<>();  
  
box.set("Payment");  
  
String value = box.get();
```

Or:

```
Box<Integer> box = new Box<>();  
  
box.set(100);  
  
Integer value = box.get();
```

T is a **type parameter**.

4. Generic Method

You can make individual methods generic.

```
public static <T> T getFirst(List<T> list) {  
    return list.get(0);  
}
```

Usage:

```
List<String> names = List.of("A", "B");  
  
String first = getFirst(names);
```

The `<T>` before the return type declares the method's type parameter.

5. Common Generic Type Names

These letters are conventions, not keywords.

```
T → Type  
E → Element  
K → Key  
V → Value  
N → Number  
R → Result
```

For example:

```
Map<K, V>
```

means key type and value type.

6. Generics in Your Spring Boot Project

You already use them constantly.

List

```
List<Payment> payments;
```

Means:

┆ This list contains Payment objects.

Map

```
Map<String, Payment> paymentMap;
```

Means:

```
String → Payment
```

Optional

```
Optional<Payment> payment;
```

Means:

┆ Either a Payment exists or the Optional is empty.

ResponseEntity

```
ResponseEntity<PaymentResponse>
```

Means:

The HTTP response body is expected to contain a `PaymentResponse`.

These are all examples of generic type safety.

7. Type Safety

Consider:

```
List<Payment> payments = new ArrayList<>();
```

The compiler ensures:

```
payments.add(new Payment()); // valid
payments.add(new Customer()); // compile error
```

This prevents a whole class of runtime `ClassCastException` problems.

Without generics:

```
List payments = new ArrayList();

payments.add(new Payment());
payments.add(new Customer());

Payment p = (Payment) payments.get(1);
```

This fails at runtime.

With generics, the error is caught much earlier.

8. Generic Types and Inheritance

This is one of the most important interview traps.

Suppose:

```
class Animal {}
class Dog extends Animal {}
```

You might think:

```
List<Dog>
```

should be a subtype of:

```
List<Animal>
```

But it is **not**.

```
List<Dog> dogs = new ArrayList<>();

List<Animal> animals = dogs; // ✗
```

Why?

Because then you could do:

```
animals.add(new Cat());
```

and the original `List<Dog>` would contain a `Cat`.

Therefore Java generics are **invariant**.

9. Wildcards

Wildcards provide flexibility when working with generic types.

Syntax:

```
?
```

Example:

```
List<?> list;
```

means:

| A List of some unknown type.

10. Unbounded Wildcard

```
List<?> list = List.of("A", "B");
```

or:

```
List<?> list = List.of(1, 2, 3);
```

You can safely read elements as `Object` :

```
Object value = list.get(0);
```

But you generally cannot add arbitrary values:

```
list.add("hello"); // ✗
```

except `null`.

Because the actual type is unknown.

11. Upper-Bounded Wildcard

```
List<? extends Number>
```

means:

▮ A list of some type that is `Number` or a subclass of `Number`.

Examples:

```
List<Integer> integers;  
List<Double> doubles;
```

Both can be passed to:

```
void printNumbers(List<? extends Number> numbers) {  
    ...  
}
```

You can read them as `Number`:

```
Number n = numbers.get(0);
```

But you generally cannot add a `Number`, because the exact subtype is unknown.

For example, if the actual list is `List<Integer>`, adding a `Double` would be unsafe.

12. Lower-Bounded Wildcard

```
List<? super Integer>
```

means:

▮ A list whose element type is `Integer` or one of `Integer`'s supertypes.

Examples:

```
List<Integer>  
List<Number>  
List<Object>
```

are valid.

This allows you to safely add an `Integer`:

```
list.add(10);
```

But reading gives you only `Object` safely:

```
Object value = list.get(0);
```

13. PECS

This is one of the **most common generics interview questions**.

PECS = Producer Extends, Consumer Super.

Producer → extends

If you're mainly **reading** values:

```
List<? extends Number>
```

The collection produces Numbers for you.

Consumer → super

If you're mainly **adding** values:

```
List<? super Integer>
```

The collection consumes Integers.

Easy example

```
void readNumbers(List<? extends Number> numbers) {  
    for (Number n : numbers) {  
        System.out.println(n);  
    }  
}
```

Reading → **extends**.

```
void addNumbers(List<? super Integer> numbers) {  
    numbers.add(10);  
    numbers.add(20);  
}
```

Adding → **super**.

Remember:

Producer → extends Consumer → super

14. ? vs T

Another common question.

?

Unknown type:

```
List<?> list;
```

You don't know what the actual element type is.

T

A named type parameter:

```
<T> T getFirst(List<T> list)
```

The method can maintain a relationship between multiple occurrences of `T`.

For example:

```
public static <T> T identity(T value) {  
    return value;  
}
```

Input and output have the same type.

So:

```
? → unknown type  
T → named type parameter
```

15. Generic Method Example

Suppose BPS has a utility that logs different response objects.

```
public <T> void logResponse(T response) {  
    System.out.println(response);  
}
```

It can accept:

```
PaymentResponse  
CustomerResponse  
TransactionResponse
```

without creating separate methods.

16. Bounded Type Parameters

You can restrict a generic type.

```
public <T extends Number> double calculate(T value) {  
    return value.doubleValue();  
}
```

Now:

```
Integer → valid  
Double  → valid  
Long    → valid  
String  → invalid
```

This gives you generic reuse while retaining type constraints.

17. `T extends Number` vs `? extends Number`

They look similar but serve different purposes.

Type parameter

```
<T extends Number>
```

Names the type so you can refer to it elsewhere.

Example:

```
public <T extends Number> T process(T value)
```

Wildcard

```
List<? extends Number>
```

means:

Some unknown subtype of Number.

This distinction is worth knowing for a 3-year interview.

18. Generic Interfaces

Example:

```
interface Repository<T> {  
    T findById(Long id);  
    void save(T entity);  
}
```

Implementation:

```
class PaymentRepository  
    implements Repository<Payment> {  
  
    @Override  
    public Payment findById(Long id) {  
        ...  
    }  
  
    @Override  
    public void save(Payment payment) {  
        ...  
    }  
}
```

Now the repository contract is strongly typed.

19. Spring Data Example

This is directly relevant to your project.

```
public interface PaymentRepository
    extends JpaRepository<Payment, Long> {
}
```

Here:

```
JpaRepository<T, ID>
  ↓
T  = Payment
ID = Long
```

So:

```
paymentRepository.findById(id);
```

returns:

```
Optional<Payment>
```

and:

```
paymentRepository.save(payment);
```

expects a `Payment`.

That's generics providing type safety throughout the repository layer.

20. Type Erasure

This is an important **3-year interview topic**.

Java implements generics largely through **type erasure**.

For example:

```
List<String>
```

and:

```
List<Integer>
```

do not become completely separate runtime generic classes.

Much of the generic type information is removed/erased during compilation, with appropriate compiler-generated casts where needed.

Conceptually:

Source code
List<String>
↓
Compilation
↓
Type checks
↓
Runtime representation largely uses raw List semantics

This is one reason you cannot normally do:

```
if (list instanceof List<String>) { }
```

because the parameterized type is not available in that form for runtime `instanceof` checking.

You can do:

```
if (list instanceof List<?>) { }
```

21. Why Can't We Do `new T()`?

With normal Java generics:

```
class Factory<T> {  
    T create() {  
        return new T(); // ✗  
    }  
}
```

The runtime doesn't know the concrete `T` in that direct form because of type erasure.

A common solution is to pass a factory/Supplier:

```
class Factory<T> {  
    private final Supplier<T> supplier;  
  
    Factory(Supplier<T> supplier) {  
        this.supplier = supplier;  
    }  
  
    T create() {  
        return supplier.get();  
    }  
}
```

Usage:

```
Factory<Payment> factory =  
    new Factory<>(Payment::new);
```

This also connects your **Generics + Functional Interfaces + Lambda** topics.

22. Raw Types

Avoid:

```
List list = new ArrayList();
```

This is a **raw type**.

Prefer:

```
List<Payment> list = new ArrayList<>();
```

Raw types weaken compile-time type safety and may produce unchecked warnings.

23. Generic Type vs Primitive Type

Generics work with reference types, not primitive types.

This doesn't work:

```
List<int> numbers; // ✗
```

Use wrapper types:

```
List<Integer> numbers; // ✔
```

Java provides autoboxing/unboxing:

```
List<Integer> numbers =  
    new ArrayList<>();  
  
numbers.add(10); // int → Integer
```

24. Common Interview Questions

Basic

1. What are generics?
2. Why were generics introduced?
3. What is type safety?
4. What is a generic class?
5. What is a generic method?
6. What does `<T>` mean?
7. What are `K`, `V`, `E` conventions?

Intermediate

8. Why is `List<Dog>` not a `List<Animal>`?
9. What is wildcard `??`?
10. What is `?` extends?
11. What is `?` super?
12. What is PECS?

13. `T` vs `??`
14. What is a bounded type parameter?
15. What is a raw type?
16. Why can't generics use primitive types?

Advanced

17. What is type erasure?
18. Why can't you do `new T()`?
19. Why can't you use `instanceof List<String>`?
20. Can static methods use class type parameters?
21. Can a generic class have multiple type parameters?
22. How do generics work with inheritance?
23. Why does Java use invariance for generic types?

25. Practical Example for BPS

Suppose you have different payment types:

```
Payment
CardPayment
CashPayment
```

You could have:

```
List<Payment> payments;
```

This gives you:

```
Only Payment objects allowed
    ↓
Compile-time safety
    ↓
No explicit casts when retrieving
```

For a repository:

```
JpaRepository<Payment, Long>
```

means:

```
Entity type = Payment
ID type     = Long
```

For an API response:

```
ResponseEntity<PaymentResponse>
```

means:

```
HTTP body type = PaymentResponse
```

So you should recognize that **generics are not a separate theoretical topic—they are**

everywhere in Spring Boot.

26. Interview-Ready Answer

If asked:

“What are generics and why are they useful?”

A strong 3-year answer:

“Generics allow us to parameterize types so the compiler can enforce type safety. For example, `List<Payment>` ensures that only `Payment` objects can be added and eliminates the need for explicit casting when retrieving them. Generics also make classes and methods reusable across different types. Java generics are invariant, so `List<Dog>` isn't a subtype of `List<Animal>`. Wildcards such as `? extends` and `? super` provide controlled flexibility, summarized by PECS—Producer Extends, Consumer Super. Java implements generics primarily through type erasure, which is why parameterized types aren't generally available for runtime type checks.”

Your priority for interviews

Focus on these in order:

Generics → type safety → T vs ? → extends → super → PECS → generic methods/classes → type erasure → raw types.

And connect them to code you already use:

```
List<Payment>
Map<String, Payment>
Optional<Payment>
ResponseEntity<PaymentResponse>
JpaRepository<Payment, Long>
```

That will make your answer sound like someone who actually works with Java/Spring rather than someone who memorized the definition of generics.